Design of "Star Seeker AI" Game Based on Q-Learning

Youtube: https://youtu.be/9PXhGcHBY7o?si=f5Cs34LAj6C2Wbpv
Github: https://github.com/justusyyr/-Star-Seeker-AI-Based-on-Q-Learning/upload/main
Directory (Assignment 1_YINYIRAN): https://drive.google.com/drive/folders/1Gdy0ScAlckKr5xlsp23hNa4oqSp_5z-B?usp=sharing

1. Introduction

This project explores how Deep Reinforcement Learning works in complex and dynamic environments. I designed and built an action game called "Star Seeker AI," where an agent controlled by the model must operate in a space filled with moving obstacles. The main goal is for the agent to learn how to find targets efficiently while avoiding collisions under strict energy limits.

The system is built with Python, using the PyTorch framework to create the Deep Q-Network (DQN) and the Pygame library to handle the environment, graphics, and physics. Unlike traditional AI that follows fixed rules, the agent in this project learns through trial and error. By pulling key features from the environment's raw data, the agent gradually develops complex decision-making skills and the ability to predict obstacle movements.

2. Game Design

2.1 Game Rules and Objectives

In "Star Seeker AI," the agent is a blue square. The map randomly creates yellow stars (targets) and purple moving meteors (obstacles). If the agent hits a purple meteor, it "dies" and the game ends. The main goal is to collect as many stars as possible to get a high score. To make the game more interesting, I added an energy system: the agent starts with 100 energy points. Every move costs 0.1 points, but catching a star adds 40 points back. This rule forces the agent to find a balance between staying safe and moving fast. If the agent only focuses on dodging meteors and ignores the stars, it will eventually run out of energy and fail.

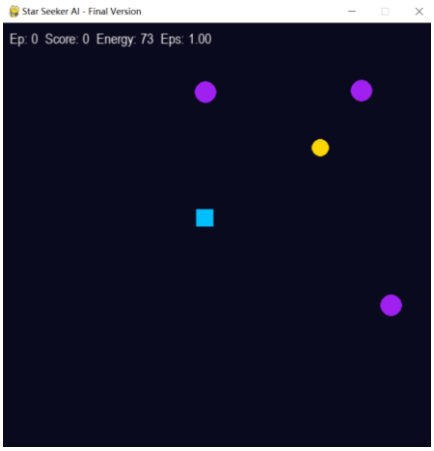

figure 1: Game Environment and User Interface

2.2 State and Action Spaces

The game environment is simplified into a 10-dimensional vector for the network input. This includes the agent's normalized coordinates, the relative position of the target, and the movement data of the three closest obstacles. The 10-dimensional state vector is composed of: 2 dimensions (agent's normalized coordinates) + 2 dimensions (relative position to target) + 3×2 dimensions (relative position of 3 closest obstacles) = 10 dimensions. This data helps the agent track the target and predict potential hits from obstacles at the same time. The action space consists of four basic moves: up, down, left, and right, allowing the agent to move freely across the screen.

2.3 Reward Function

The reward system uses a mix of different signals. Collecting a star gives a +20 reward, which is the core base reward and overrides the distance-based guiding reward, ensuring the agent takes star collection as its primary behavior, while hitting an obstacle or going off-screen results in a -15 or -10 penalty. I also used a "Guiding Reward" based on distance to help the agent learn faster. It gets a small positive feedback for every step it takes toward the target and a small penalty for moving away. This method effectively solves the "sparse reward" problem where the agent might otherwise struggle to find the goal.

3. Q-Learning Implementation

3.1 Network Architecture

I built a four-layer Multi-Layer Perceptron (MLP) for this project. The network structure follows a 10➔256➔128➔64➔4 design. It uses the ReLU activation function to help the model learn non-linear patterns. The final layer outputs Q-values for the four possible moves.

Specifically speaking, I used a Deep Q-Network (DQN) to estimate the Q-function. To give the agent better decision-making skills, I built a neural network that acts as its "brain." This network consists of four fully connected layers. It starts by taking in 10-dimensional data from the game, such as the ship's position and the distance to obstacles. This information then passes through two hidden layers—one with 256 neurons and another with 128—to process the game's logic and pull out key features. Finally, the output layer provides a score for each of the four possible moves: up, down, left, and right. Through this network, the agent can analyze dangerous situations and pick the path that offers the highest reward.

```python
#1. DQN Architecture
class DeepQNet(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DeepQNet, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, output_dim)
        )

    def forward(self, x):
        return self.network(x)
```

figure 2: Related Code

3.2 Training Strategy and Hyperparameters

For the training strategy, I used an Experience Replay buffer. Since data in reinforcement learning is usually highly correlated, I set up a buffer with a capacity of 20,000 to store the agent's past experiences. During training, the model randomly picks a mini-batch of 64 from this buffer to break the data correlation, which helps the network stay stable as it learns.

I also used an epsilon-greedy strategy to balance exploration and exploitation. At the start, I set epsilon to 1.0 to force the agent to try random moves. Over time, this value drops to 0.05, letting the agent rely more on the neural network's decisions once it becomes smarter. Epsilon is decayed exponentially with a decay rate of 0.996 per episode, starting from 1.0 and converging to a minimum of 0.05. Other core settings include a learning rate of 0.0005 (using the Adam optimizer) and a discount factor (gamma) of 0.98, which helps the agent value long-term rewards.

4. Evaluation and Challenges

4.1 Performance Evaluation

Over 600 training episodes, the agent showed a clear learning path from simple exploration to smart decision-making. It reached a peak score of 33 at Episode 140. This result proves that the DQN structure is very effective for dynamic search tasks. It also shows that the Guiding Reward I added really helped the agent learn much faster in the early stages.

Between episodes 150 and 400, the agent's performance leveled off, with scores staying between 7 and 10 points. This shows that the agent learned how to dodge moving obstacles while also focusing on finding targets. Near the end of the training, some scores dropped to 0 because of random "dead ends”situations where the meteors blocked all possible paths at once. However, the agent's overall ability to make decisions remained reliable despite these environmental challenges.

```
Episode 0 | Final Score: 0
Episode 10 | Final Score: 0
Episode 20 | Final Score: 1
Episode 30 | Final Score: 2
Episode 40 | Final Score: 1
Episode 50 | Final Score: 1
Episode 60 | Final Score: 3
Episode 70 | Final Score: 1
Episode 80 | Final Score: 0
Episode 90 | Final Score: 11
Episode 100 | Final Score: 3
Episode 110 | Final Score: 7
Episode 120 | Final Score: 6
Episode 130 | Final Score: 6
Episode 140 | Final Score: 33
Episode 150 | Final Score: 1
Episode 160 | Final Score: 1
```

figure 3

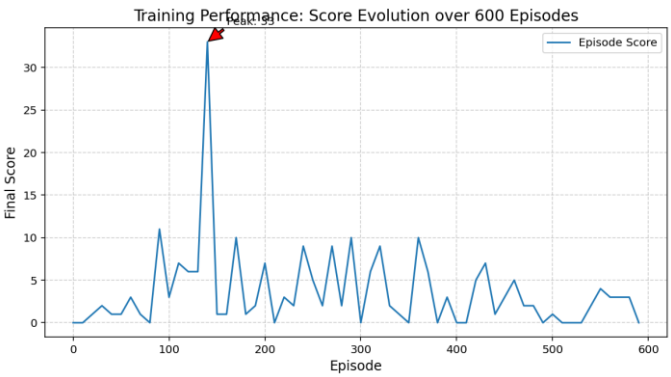

figure 4: Training Performance and Score Evolution

4.2 Challenges and Solutions

During development, the biggest challenge was the agent's tendency toward "passive avoidance." At first, because the penalty for crashing was much higher than the random

reward of collecting a star, the agent often chose to stay still in a corner to avoid risks. To fix this, I added the Guiding Reward mentioned earlier. I also adjusted the speed of the obstacles so they moved slower in the beginning. This helped the AI build "confidence" in a simpler environment before I moved the speed back to normal for the final training. This approach allowed the model to converge smoothly.

```
        #CHALLENGE 1: Guiding Reward (Potential Reward)
        current_dist = np.linalg.norm(self.player_pos - self.target_pos)
        if current_dist < self.prev_dist:
            reward += 1.0  # Reward for moving closer
        else:
            reward -= 1.0  # Penalty for moving away
        self.prev_dist = current_dist
```

figure 5: Related Code

5. Conclusion

This project shows that Deep Reinforcement Learning can work well in a fast-paced action game. By using careful Reward Shaping and a well-planned state space, the agent learned more than just how to stay alive. It also developed a smart strategy for collecting targets efficiently.

The test data shows that the Guiding Reward I added helped fix the "sparse reward" problem, which let the model start learning much faster in the beginning. Even though the agent sometimes got stuck when obstacles moved in a way that blocked all paths, its high score of 33 at Episode 140 proves that this DQN setup has real potential for finding paths in changing environments.

Overall, this work confirms that the DQN structure can handle tasks that require balancing multiple goals at once. In the future, I plan to work on making the agent's decisions even more stable when the game environment becomes extremely difficult.

6. Reference

[1] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533.

[2] Pygame Developers. (2024). *Pygame Library Documentation*. https://www.pygame.org/docs/

[3] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*.

[4] KI-cheng. (2025). *QlearningTankWar: ML project*. GitHub. https://github.com/KI-cheng/QlearningTankWar (Course Reference: QlearningTankWar ML Project).